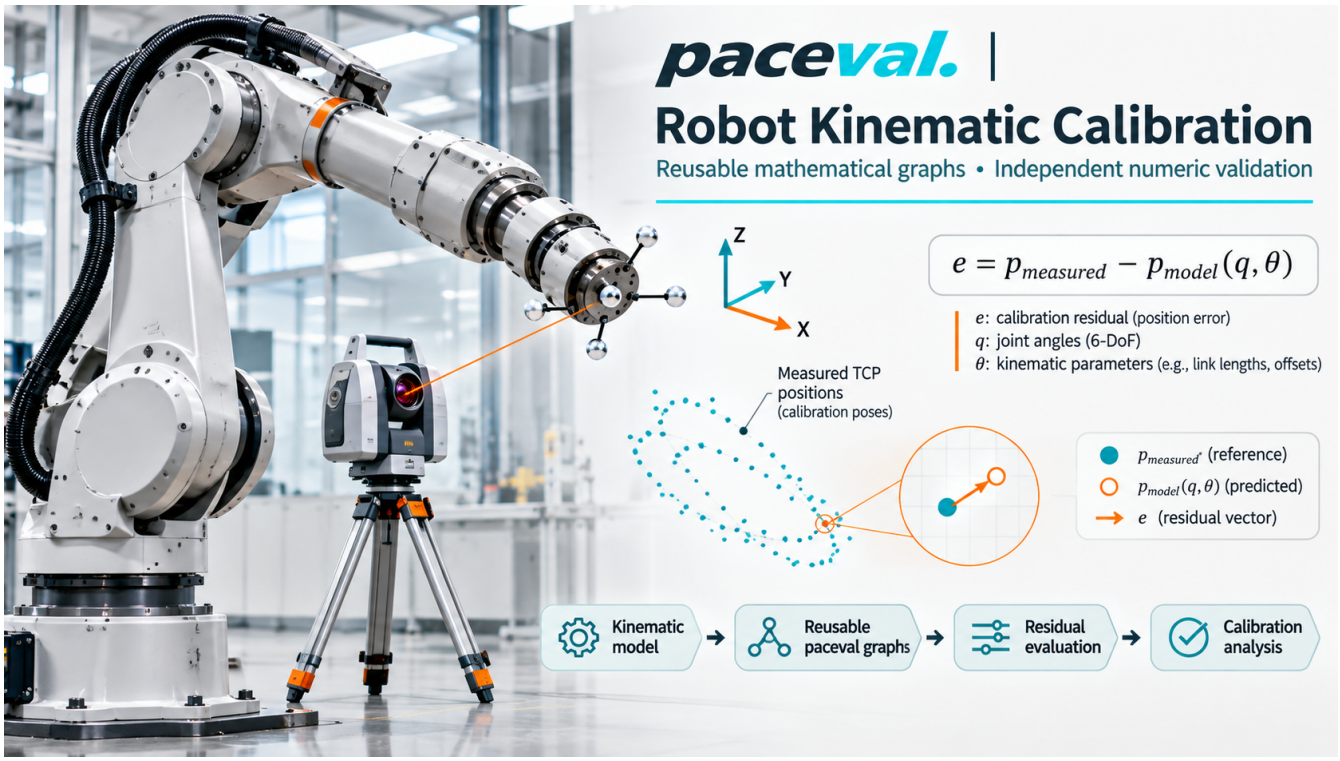
Parameterized Robot Calibration Residuals

Reusable mathematical execution graphs in a six-joint, position-only kinematic calibration workflow



Illustrative robotic metrology scene. No real robot or measurement instrument was operated.

Research result. Six reusable paceval graphs computed three Cartesian coordinates and three residuals for an illustrative six-revolute-joint (6R) arm. A separate damped least-squares C++ solver identified nine synthetic kinematic corrections from 120 measured positions and improved prediction on 48 independent positions.

Holdout RMSE before	8.8567 mm
Holdout RMSE after	0.01780 mm
Independent precision check	1,008 / 1,008 residual components passed
Raw TINC audit	24 / 24 sampled values enclosed

Original paceval 4.25 Linux runtime | GCC 14.2 and Clang 17 | Published source and test data | No performance comparisons

1. Research question and experimental architecture

Can reusable mathematical expression graphs evaluate three-dimensional robot kinematics and signed residual components over many measurement poses, while preserving numerical fidelity and supporting independently checked calibration parameters? This study converts the WP-016 research proposal into a controlled executable experiment.

The original proposal illustrates a two-link planar squared x-coordinate error. We extend it to a six-revolute-joint open chain with measured Cartesian TCP positions, nine estimated parameters, a fixed solver, a distinct held-out pose set and a deliberate non-identifiability control.

Research workflow: reusable equations and independent calibration logic

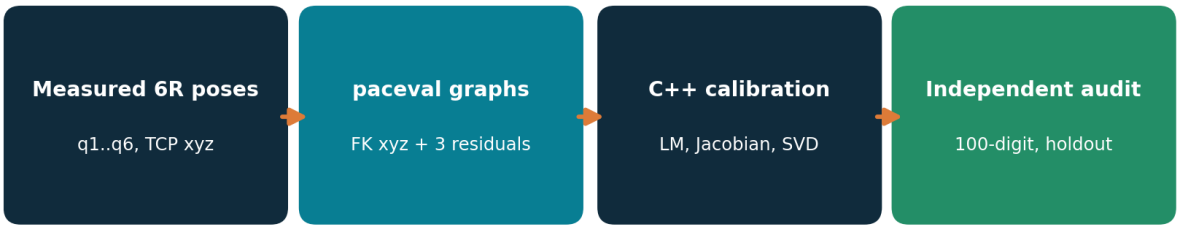


Figure 1. Precise separation between persistent paceval mathematics, host-owned optimizer, and independent numerical and observability audits.

Scope of calibration

The estimated parameter vector is (o1,...,o6, delta_l2, delta_l3, delta_l6). Joint-zero offsets are in radians and link corrections are in metres. The base frame, tool frame and all other link lengths are fixed by the synthetic model. This is position-only calibration; it is not a full six-component pose or commercial-robot calibration.

Physics versus numerical correctness

Recovering known injected model corrections is a valid software verification, but does not establish model adequacy for a physical robot. Sensor traceability, kinematic frames, unmodeled compliance and real measurement noise would require another study.

2. Six-axis kinematic model and observations

Six successive intrinsic local rotations follow the sequence Z-Y-Y-X-Y-Z, with translations after each joint rotation. Links 1 and 4 translate along their local z-axis and links 2, 3, 5 and 6 along their local x-axis. Nominal link lengths are (0.45, 0.53, 0.42, 0.16, 0.27, 0.14) metres.

For each observation i, measured TCP coordinate vector m_i is compared with the model f(q_i,theta). paceval evaluates each signed residual r_i = m_i - f(q_i,theta) in metres. Three coordinate expressions and three signed residual expressions are generated symbolically and provided as persistent calculation objects.

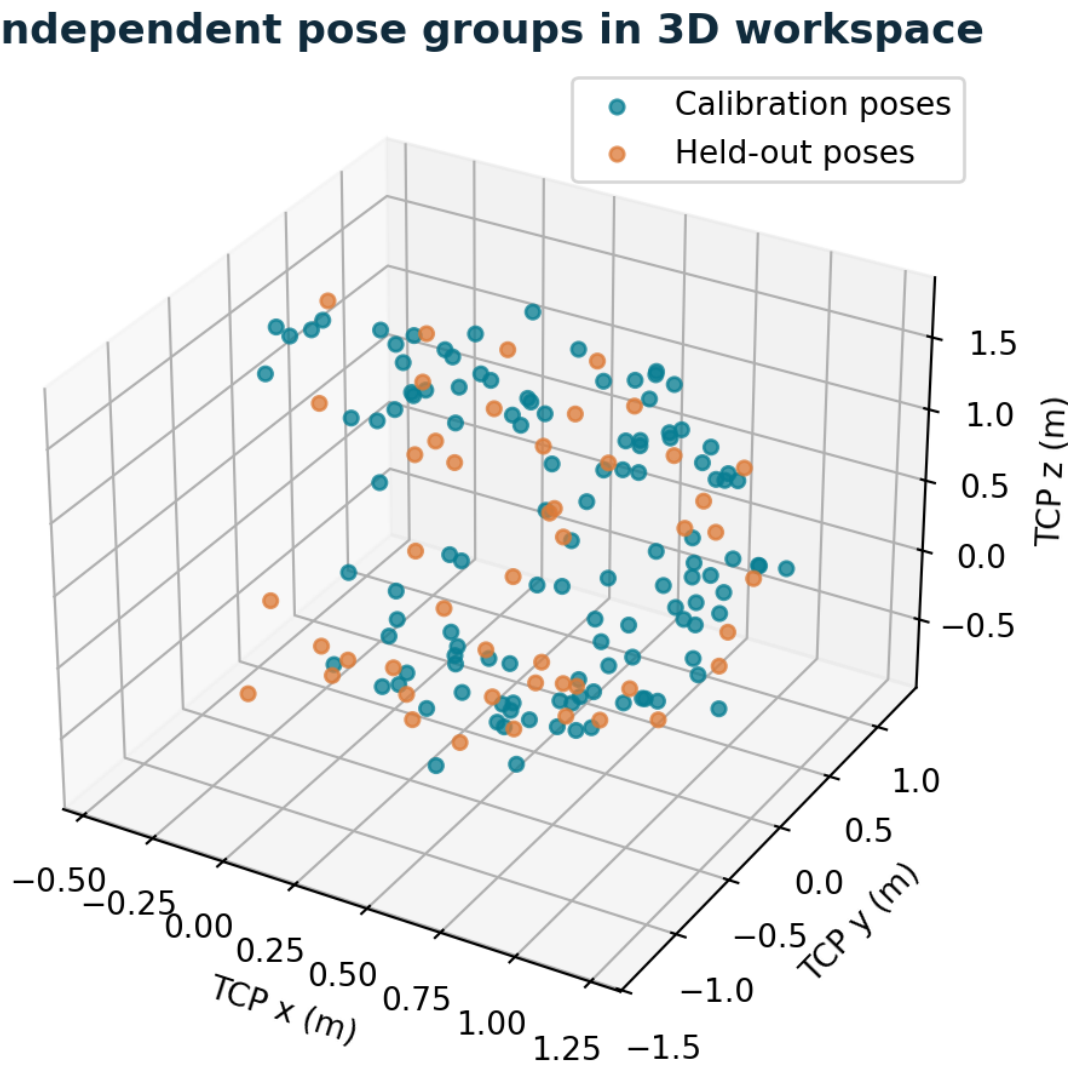


Figure 2. Independent training and withheld measurement configurations in the three-dimensional synthetic workspace.

Dataset construction

A fixed-seed deterministic generator builds 120 diverse calibration poses and 48 independent holdout poses. Known parameter offsets and bounded deterministic coordinate perturbations generate the measurements. A further 24 repeated poses deliberately lack parameter-identifying information. Complete double-precision CSV inputs are shipped with the program.

3. Direct SDK integration and solver

The project compiles the original SDK forwarding sources paceval_main.h and paceval_main.cpp and links the separately supplied Linux engine. paceval_CreateComputation creates three FK position graphs, three residual graphs, and a separate interval-mode control graph; the computation handles remain alive for the entire calibration.

For each parameter iterate, paceval_dGetMultipleComputationsResultsExt calculates residual components in batches. A central finite-difference scheme differentiates these paceval residuals with respect to the nine candidate parameters. The C++ host then forms J^T J and J^T r, solves a damped normal equation and accepts bounded parameter updates only if the sum of squared residuals decreases.

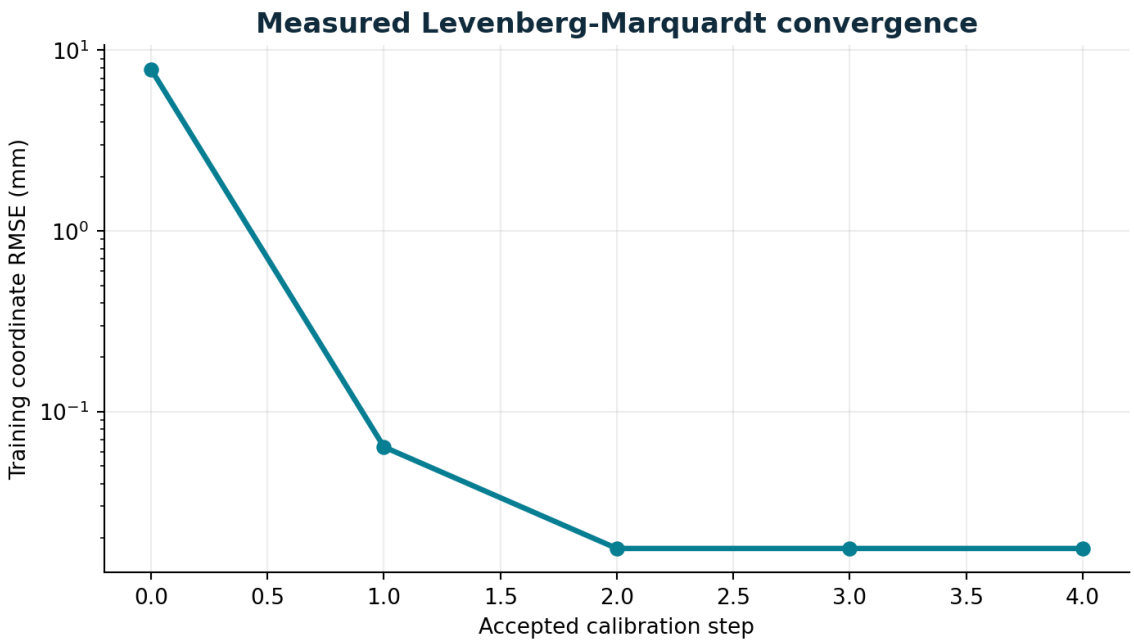


Figure 3. Actual convergence from the nominal model to the calibrated parameters in four accepted iterations.

Mathematical responsibility

The native reference model uses explicit 3x3 matrix rotations and translations; it is employed only for synthetic observation generation and independent checks. The optimizer never replaces paceval-calculated residuals with that native model. The mathematical expressions are generated into a checked-in header; Python and SymPy are not needed to run the C++ program.

4. Independent holdout result

Parameter fitting uses only the 120 calibration poses. After fitting, both the uncalibrated and calibrated models predict 48 entirely separate joint configurations. The coordinate-level RMSE decreases from 8.8567 mm to 0.01780 mm. The final residual is close to the synthetic measurement noise envelope and is not an industrial positioning specification.

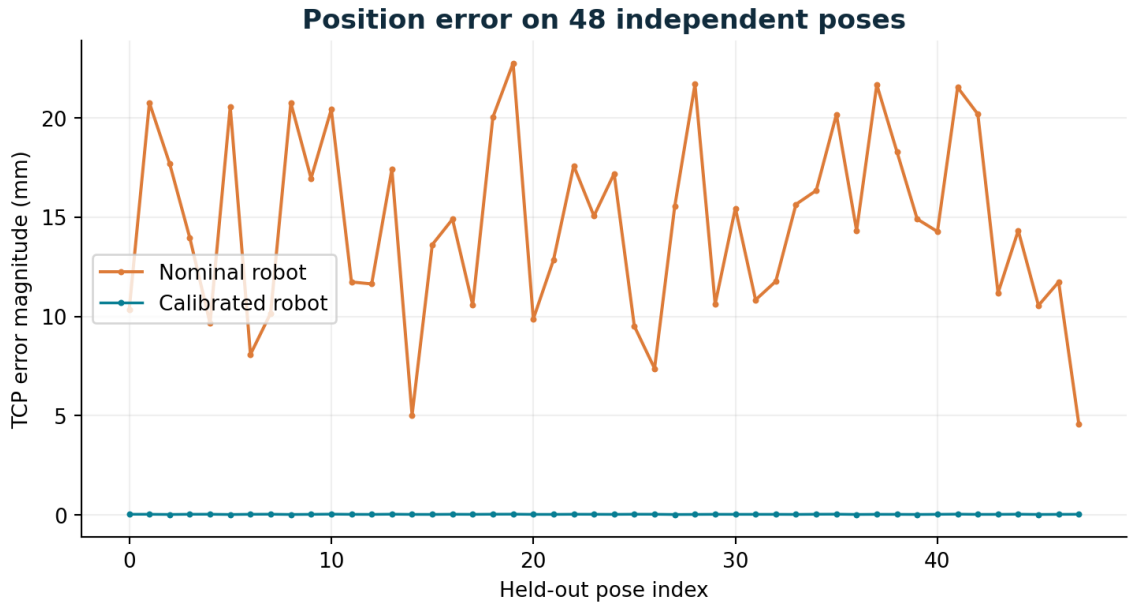


Figure 4. The measured-vs-predicted TCP residual magnitude for every independent holdout configuration, before and after calibration.

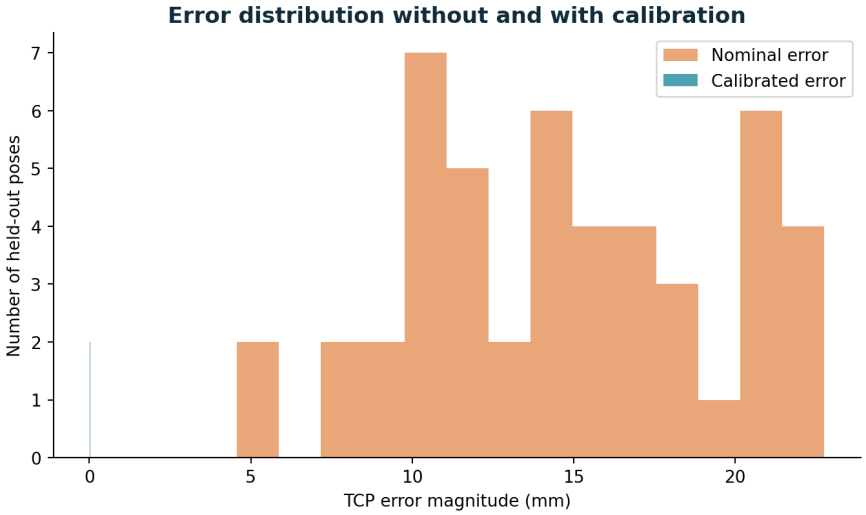


Figure 5. Distributions of nominal and calibrated TCP errors on the independent holdout set.

5. Parameter recovery and identifiability

All nine injected corrections are recovered to a maximum absolute error of 9.533e-06 in their respective radian/metre units. The independently calculated sensitivity matrix has a minimum singular value of 1.2766 for the diverse training poses.

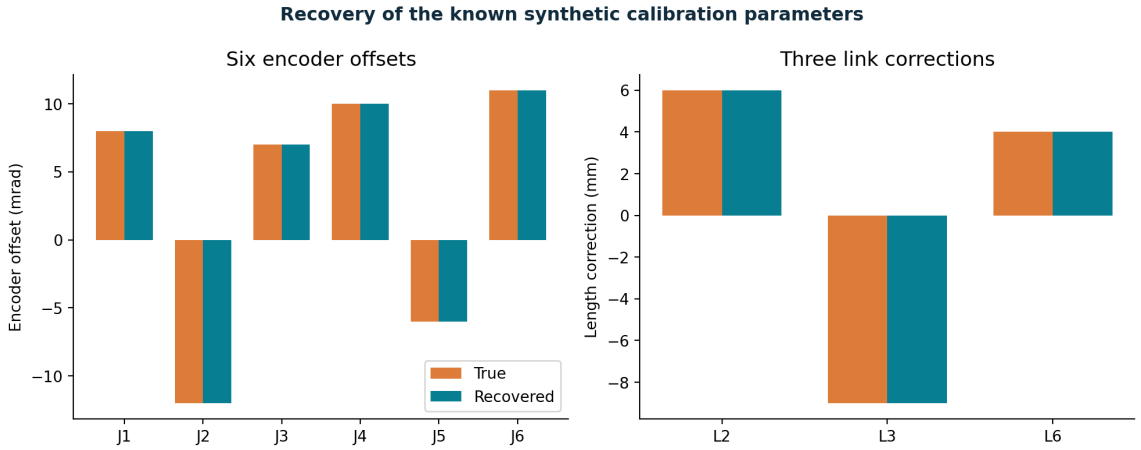


Figure 6. Known synthetic corrections against fitted zero offsets and link-length changes.

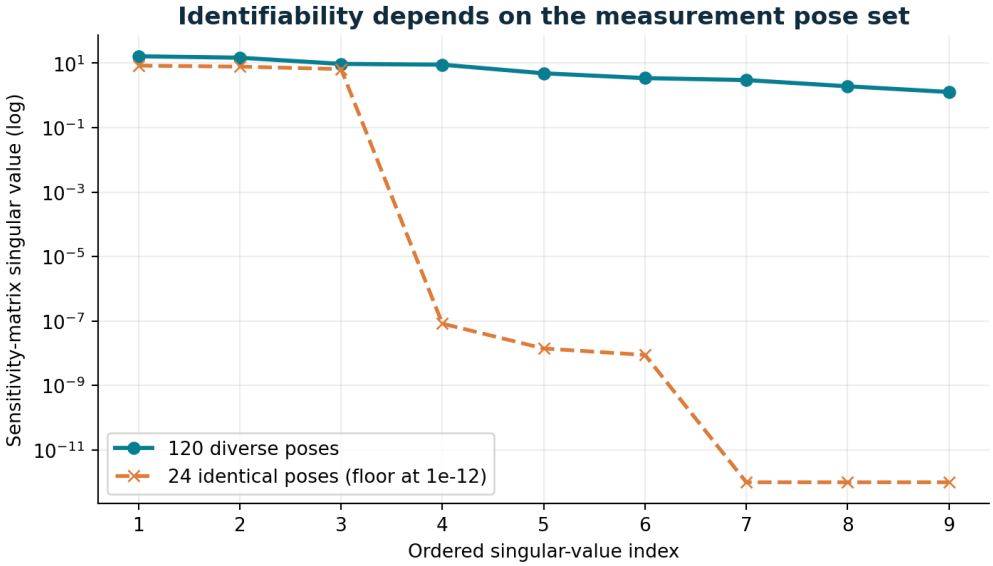


Figure 7. The full calibration-pose set has numerical column rank nine; 24 identical poses do not. Values at 1e-12 on the plot are a display floor for zero singular values.

6. Independent 100-digit arithmetic verification

An independent Python implementation evaluates the same six-joint sequence as 3x3 high-precision matrix products, rather than importing paceval formulas or trusting native C++ test outputs. All 1,008 normal-model and calibrated-model residual components over 168 poses lie within the predeclared numerical tolerance.

Maximum absolute reference difference: 6.252e-16 m across the complete independent audit. A separate NumPy SVD confirms the well-conditioned and rank-deficient pose designs. No numerical error claim is extended beyond the tested input domain and environment.

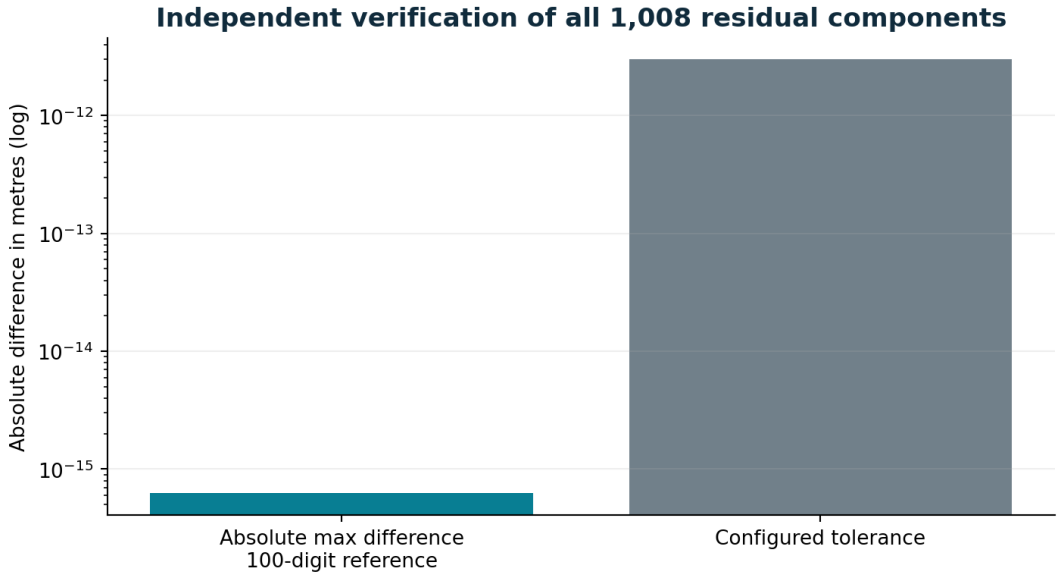


Figure 8. High-precision maximum residual difference compared with the configured acceptance tolerance.

TINC pointwise subexpression experiment

Twenty-four distinct synthetic inputs were evaluated for the smooth kinematic subexpression l2*cos(q2+o2). Raw TINC lower and upper outputs were compared against 100-digit evaluations of the actually passed binary64 inputs: 24 of 24 contained the reference. This is a sample-wise numerical observation, not a global interval guarantee for the full kinematic model.

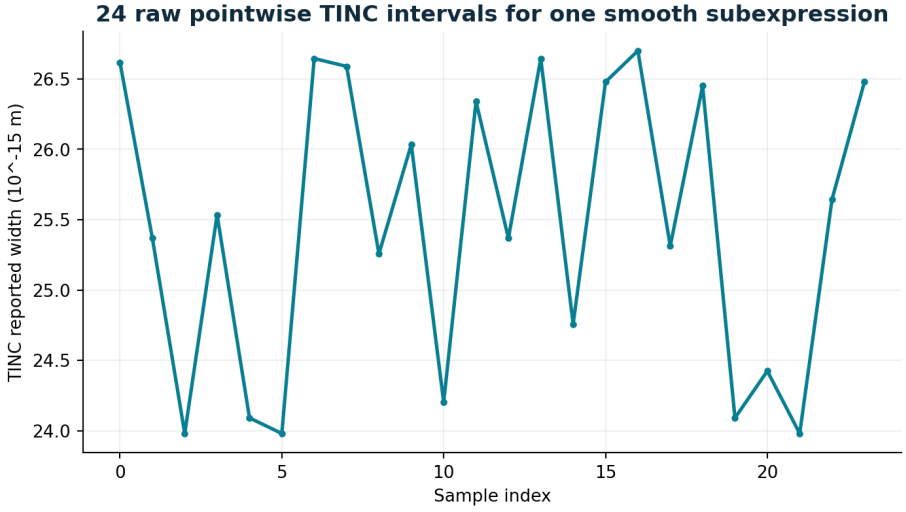


Figure 9. Widths of the 24 raw, unmodified TINC intervals for the audited kinematic subexpression.

7. Reproducibility and quality assurance

The GitHub archive contains a standalone CMake build in src/, the original SDK forwarding sources, the expanded symbolic coordinate and residual graphs, the synthetic input CSVs, the full parameter and holdout results, an independent high-precision audit, image-generation scripts and publication figures.

Numerical kernel	6 ordinary paceval graphs plus one TINC graph
Calibrated parameters	6 encoder offsets (radians) and 3 length corrections (metres)
Training / holdout / degenerate	120 / 48 / 24 pose records
High precision	1,008 coordinate residual comparisons, 100 decimal digits
Reproducibility	Validated fixed-seed data generation and CSV roundtrip
Input safety	Nonfinite coordinates, joint range errors and split mismatch rejected
Test matrix	GCC, Clang, and ASan/UBSan builds; CTest and CSV replay

Minimal Linux reproduction

From repository root, with an independently supplied, compatible original Linux .so:

```
export PACEVAL_LIB=/absolute/path/to/libpaceval_linux_sharedLIB.so
cmake -S src -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build --parallel
ctest --test-dir build --output-on-failure
./build/robot_calibration --out results --batch 128
python3 src/validate_outputs.py --examples results
```

Reproduction boundary

The executable requires C++17 and the original paceval engine; the scientific Python audit requires numpy and mpmath, and the optional visualizations and expression regeneration additionally require matplotlib and sympy. The repository does not redistribute the separately licensed original Linux engine binary.

8. Interpretation, applications and sources

Technical result. This implementation demonstrates that a non-trivial, 6-axis robot kinematic residual model can be compiled to persistent paceval graphs, reused across observations and parameter iterations, embedded within a host-owned nonlinear calibration solver and checked against independently calculated high-precision results. It additionally demonstrates why pose diversity matters for parameter identifiability.

Research next steps. Extend position-only residuals to orientation residuals on SO(3), model realistic joint-axis parameter changes, introduce sensor covariance and robust outlier handling, select poses to improve observability, and test against independently collected laser-tracker measurements. Independent field validation is necessary before any claim of hardware-specific absolute positioning accuracy.

Selected primary literature and documentation

Lynch, K.M. and Park, F.C., Modern Robotics: Mechanics, Planning, and Control. Northwestern University, chapters on forward kinematics and robot Jacobians. https://modernrobotics.northwestern.edu/

Okamura, K. and Park, F.C., Kinematic calibration using the product of exponentials formula. Robotica (2009). https://www.cambridge.org/core/journals/robotica/article/kinematic-calibration-using-the-product-of-exponentials-formula/8D1CCCF7F714A624B26FE37042432B5A

Kang et al., Two-step calibration of 6-DOF industrial robots by grouping kinematic parameters based on distance constraints. Measurement (2024), 114906. https://www.sciencedirect.com/science/article/pii/S0263224124007917

paceval mathematical engine: original C/C++ API and computation objects. https://paceval.com/api/

Transparency of evidence

All reported success and residual statistics are generated by this synthetically specified demonstrator. The hero graphic is illustrative and is not a photograph of a measurement conducted in this study. No speedup, interval certification, commercial robot validation, safety qualification or production deployment is claimed. The underlying SDK retains its original copyright and AGPL notices; no supplementary license notice file is added.

Research release: November 12, 2026. Whitepaper and all figures are reproducible using the included input data and scripts.